

ASSIGNMENT 1: CREATE A README FILE FOR THE INGESTION UI DEVELOPED IN RAG

The README file was generated using the Copilot plugin for VS CODE. It describes the application's architecture and what each component does. Below is the content of the README file

RAG Mongo Demo

A demo RAG (Retrieval-Augmented Generation) pipeline for managing test cases and user stories. It lets you upload Excel exports, convert them to JSON, generate vector embeddings with Mistral AI, store everything in MongoDB Atlas, and then query it back using vector search, BM25 keyword search, hybrid search, and LLM-based reranking (Groq).

The project is a classic two-process local dev setup:

- **client** — a Create React App (React 19 + MUI) single-page app, served on `http://localhost:3000`
- **server** — an Express API (`server/index.js`), served on `http://localhost:3001`

The two only talk to each other over HTTP — the client is a pure REST client with a hardcoded `API_BASE = 'http://localhost:3001/api'` in every component (see `client/src/components/`). There is no server-side rendering and no shared code between client and server at runtime.

...

Browser (React SPA, :3000)

| axios / fetch → `http://localhost:3001/api/*`



Express API (`server/index.js`, :3001)

| spawns short-lived Node scripts, calls Mistral/Groq, talks to MongoDB



MongoDB Atlas (vector store) Mistral AI (embeddings) Groq (rerank/summarize)

...

Running it

```
bash
npm install      # installs server deps, postinstall installs client deps too
npm run dev      # concurrently runs `node server/index.js` (:3001) and `react-scripts start` (:3000)
```

Configuration lives in `.env` at the project root (see `.env.example`): MongoDB connection string, database/collection/index names, and the Mistral/Groq API keys. The `/api/env` endpoint (used by the **Settings** tab) can read and overwrite this file at runtime.

How the frontend is organized

`client/src/App.js` renders a fixed sidebar of tabs (no client-side routing — just component swapping via `useState`), each backed by one component in `client/src/components/`:

Tab	Component	Talks to
Convert to JSON	<code>data/ConvertToJson.js</code>	<code>POST /api/upload-excel</code>

```
| Embeddings & Store | `data/EmbeddingsStore.js` | `GET /api/files`, `POST /api/create-embeddings-batch`, `GET /api/jobs/:id` |
| Query Preprocessing | `processing/QueryPreprocessing.js` | `POST /api/search/preprocess`, `/analyze`, `/deduplicate` |
| Vector Search | `search/QuerySearch.js` | `POST /api/search` |
| BM25 Search | `search/BM25Search.js` | `POST /api/search/bm25` |
| Hybrid Search | `search/HybridSearch.js` | `POST /api/search/hybrid` |
| Score Fusion | `search/RerankingSearch.js` | `POST /api/search/rerank` |
| Summarize & Dedup | `processing/SummarizationDedup.js` | `POST /api/search/summarize` |
| Prompt & Schema | `processing/PromptSchemaManager.js` | `POST /api/test-prompt` |
| Settings | `settings/Settings.js` | `GET/POST /api/env` |
```

Every component does its own `axios`/`fetch` calls directly — there's no shared API client or global state manager (no Redux/Context for server data). Each component owns its own `useState` for loading/error/result, and uses `notistack` for toast notifications.

The core pipeline: Excel → JSON → Embeddings → MongoDB

This is the flow the "Convert to JSON" and "Embeddings & Store" tabs implement together, and it's the heart of the app.

1. Upload & convert (ConvertToJson.js → POST /api/upload-excel)

1. The user picks an `.xlsx/.xls` file, a **Data Type** (testcases or userstories), and a sheet name in the UI.
2. On submit, the client builds a `multipart/form-data` request (file, sheetName, dataType) and posts it to `POST /api/upload-excel`.
3. On the server (server/index.js:271), `multer` saves the upload to `uploads/` with a timestamped filename.
4. The handler **writes a throwaway Node script to disk** (temp-excel-convert-*.js) that uses the `xlsx` package to:
 - read the requested sheet (falling back to the first sheet if the name doesn't match),
 - map raw Excel column headers to normalized JSON keys via a `columnMap` (different maps for test cases vs. user stories — e.g. `"Test ID" → "id"`, `"Story Points" → "storyPoints"`),
 - write the result as an array of JSON objects to `src/data/converted-<timestamp>.json` (test cases) or `src/data/stories-<timestamp>.json` (user stories).
5. The server `spawn()`s that script as a child process, streams its stdout/stderr, waits for it to exit (30s timeout), then deletes both the temp script and the original upload.
6. The response (`{ success, outputFile, output }`) is shown back in the UI's "Conversion Results" card, and the new file now appears in `src/data/` ready for the next step.

Running the conversion as a spawned child script (rather than requiring it inline) is a deliberate isolation choice: it keeps the long-running `xlsx` parsing off the main Express event loop and gives each conversion a clean, disposable process.

2. Generate embeddings & ingest into MongoDB (EmbeddingsStore.js)

1. On mount, the tab calls `GET /api/files` to list every `*.json` file in `src/data/` (the outputs from step 1), and `GET /api/jobs/active` to resume any embedding job that was already running (e.g. after a page refresh).
2. The user selects one or more files in the MUI `DataGrid` and clicks **Create Embeddings**. The client auto-detects the job type from the filename (testcase/test-case → test cases, stor/story → user stories) and picks the matching batch script name (create-embeddings-batch-mistral.js or create-userstories-embeddings-batch-mistral.js).

3. It posts `{ files, scriptName, jobType }` to `POST /api/create-embeddings-batch`.
4. The server (`server/index.js:776`) first validates that the target Mongo database/collection exist, then immediately creates an in-memory ****job**** (jobs` Map, `server/index.js:50`) and returns `{ jobId }` — the actual work happens asynchronously so the HTTP request doesn't hang for minutes.
5. `processBatchEmbeddings()` spawns the chosen script from `src/scripts/embeddings/`, passing the selected filenames via the `EMBEDDING\_INPUT\_FILES` env var. Each script (see `src/scripts/embeddings/create-embeddings-batch-mistral.js`):
 - reads the input JSON file(s),
 - chunks records into batches of 50 and calls ****Mistral AI's embeddings API**** (src/scripts/utilities/mistralEmbedding.js, model `mistral-embed`, 1024-dim vectors) with up to 3 concurrent batch calls (via `p-limit`) and retry/backoff,
 - attaches the resulting vector plus `embeddingMetadata` (model, cost, tokens) to each record,
 - inserts the enriched documents into MongoDB in sub-batches of 100 via `collection.insertMany()`.
6. As the child process writes to stdout, the server updates the job's `status`, `progress`, and `currentFile` in the jobs` Map.
7. The client polls `GET /api/jobs/:jobId` every 2 seconds and renders a live progress bar until `status === 'completed'`, then shows a success/failure summary per file.

At the end of this pipeline, every test case / user story is a MongoDB document shaped roughly like:

```

```jsonc
{
 "module": "...", "id": "...", "title": "...", "description": "...", "steps": "...",
 // ...other original fields...
 "embedding": [0.0123, -0.0456, /* 1024 floats */],
 "embeddingMetadata": { "model": "mistral-embed", "provider": "mistral-ai", "cost": 0.000004,
"tokens": 42 },
 "sourceFile": "converted-1234567890.json",
 "createdAt": "2026-...Z"
}
```

```

which is exactly what the vector search endpoint queries against.

3. Querying it back

MongoDB Atlas needs a ****Vector Search index**** (defined in `src/config/testcases-vector-index.json` — cosine similarity over the 1024-dim `embedding` field, plus filter fields for `id`/`module`/`title`/etc.) and, for keyword search, a ****BM25/text index**** — these are created once in Atlas and referenced by name via `VECTOR\_INDEX\_NAME` / `BM25\_INDEX\_NAME` in `.env`.

- ****Vector Search**** (QuerySearch.js → `POST /api/search`): embeds the query text with Mistral, then runs a `\$vectorSearch` aggregation pipeline (numCandidates ≈ 10× the requested limit, cosine similarity), optionally post-filtered by metadata (module, priority, etc.) via `\$match`.
- ****BM25 Search**** (BM25Search.js → `POST /api/search/bm25`): pure keyword/full-text search using Atlas Search's text index — no embedding call needed.
- ****Hybrid Search**** (HybridSearch.js → `POST /api/search/hybrid`): combines vector and BM25 results.
- ****Score Fusion / Reranking**** (RerankingSearch.js → `POST /api/search/rerank`): fuses BM25 + vector scores and/or reranks candidates using Groq (src/scripts/utilities/groqClient.js, `rerankDocuments()`).

- ****Summarize & Dedup**** (SummarizationDedup.js → `POST /api/search/summarize`, `/api/search/deduplicate`): also backed by Groq (summarizeResults()), used to collapse near-duplicate results and produce a natural-language summary.
- ****Query Preprocessing**** (QueryPreprocessing.js → `POST /api/search/preprocess`, `/analyze`): normalizes, expands abbreviations, and expands synonyms in the raw query before it's embedded (src/scripts/query-preprocessing/).

API reference (server/index.js)

| Method & Path | Purpose |
|---|---|
| GET /api/health | Liveness check |
| GET /api/jobs/active / GET /api/jobs/:jobId | Poll background job status |
| GET /api/files | List convertible JSON files in `src/data/` |
| GET /api/metadata/distinct | Distinct `module`/`priority`/`risk`/`type` values for search filter dropdowns |
| POST /api/upload-excel | Upload Excel → convert to JSON (step 1) |
| POST /api/create-embeddings | Legacy per-file embedding (single Mistral call per record) |
| POST /api/create-embeddings-batch | Batch embedding + Mongo ingestion (step 2, used by the UI) |
| GET /api/env / POST /api/env | Read/write `.env` (Settings tab) |
| POST /api/search | Vector search |
| POST /api/search/bm25 | Keyword/BM25 search |
| POST /api/search/hybrid | Combined vector + BM25 |
| POST /api/search/rerank | Score-fusion + Groq reranking |
| POST /api/search/preprocess, /analyze | Query normalization/expansion |
| POST /api/search/deduplicate, /summarize | Groq-based dedup & summarization |
| POST /api/test-prompt | Prompt/schema testing (Prompt & Schema tab) |
| POST /api/search/user-stories | Search over the user-stories collection |
| GET /api/testcases/latest-id | Next available test case ID |

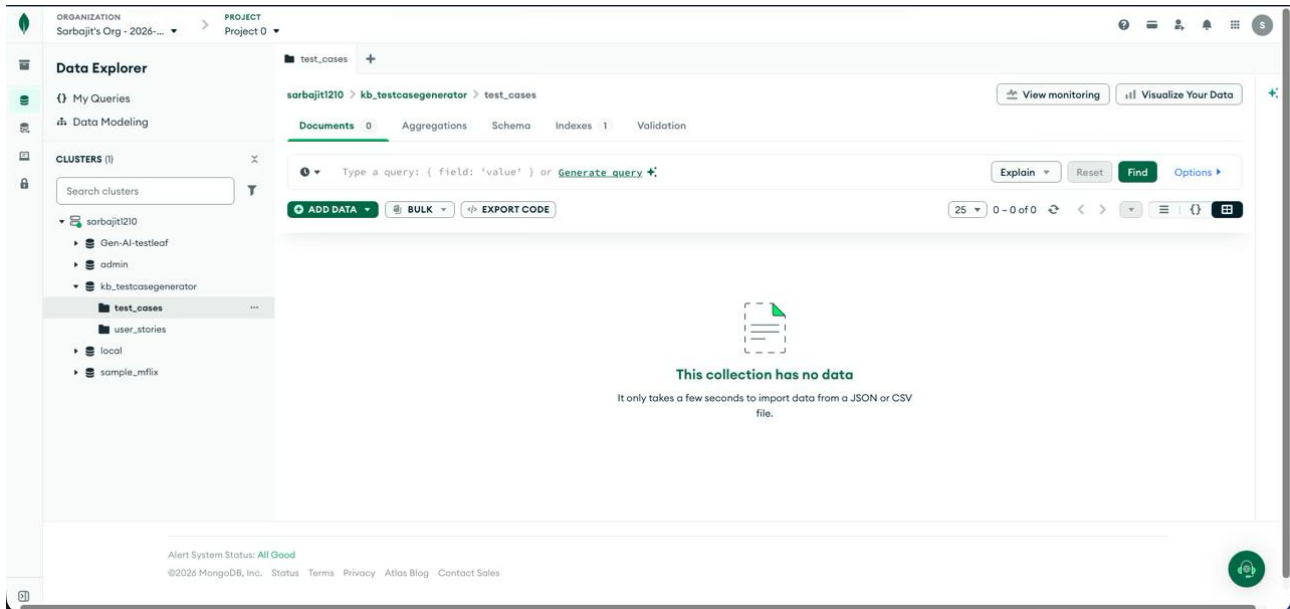
Notes

- Both the Excel→JSON conversion and the embedding/ingestion steps run as spawned child Node processes rather than inline in the Express handler, so a slow parse or a long embedding run can't block other API requests.
- Embedding jobs are tracked in an in-memory `Map` (jobs), not a database — job history is lost on server restart, though an in-progress job survives a ****browser**** refresh (the client re-polls `GET /api/jobs/active`).
- `.env` contains live credentials (MongoDB URI, Mistral/Groq API keys) and is not committed to git — copy `.env.example` and fill in your own values.

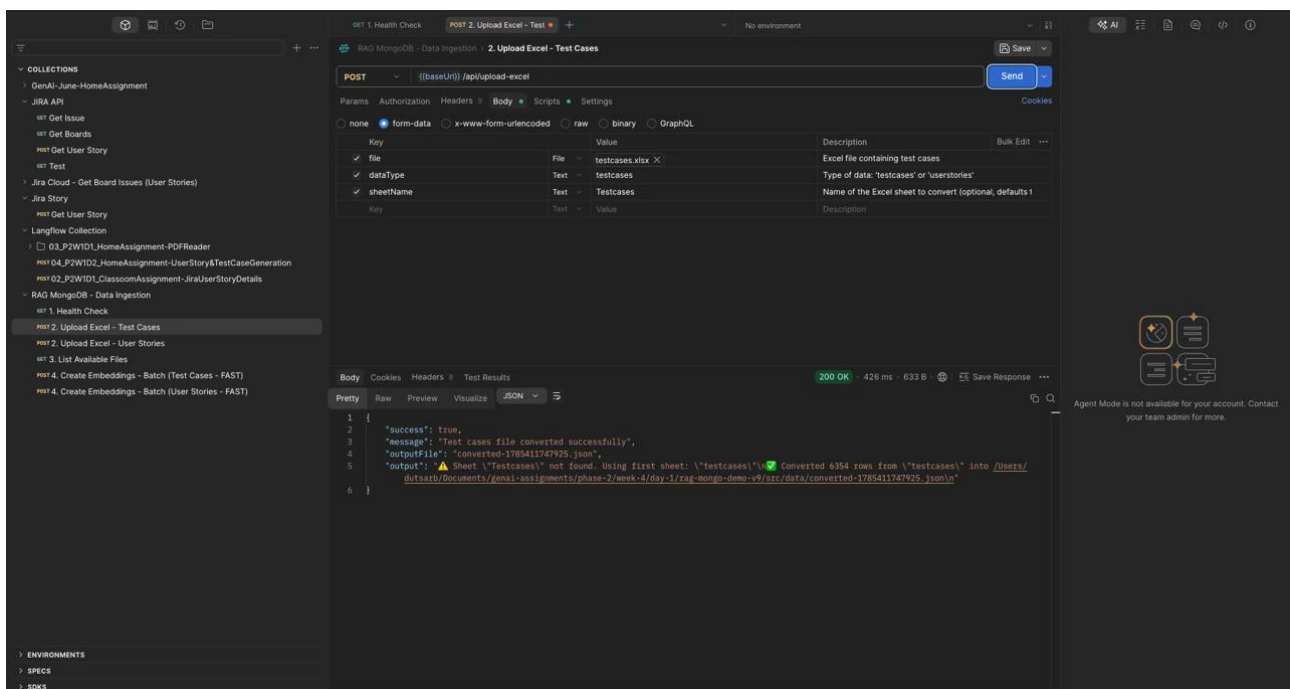
ASSIGNMENT 2: EXECUTE INGESTION PIPELINE USING POSTMAN API COLLECTION. DATA INGESTION INTO *test_cases* AND *user_stories* COLLECTION

1. INGESTION TO *test_cases*

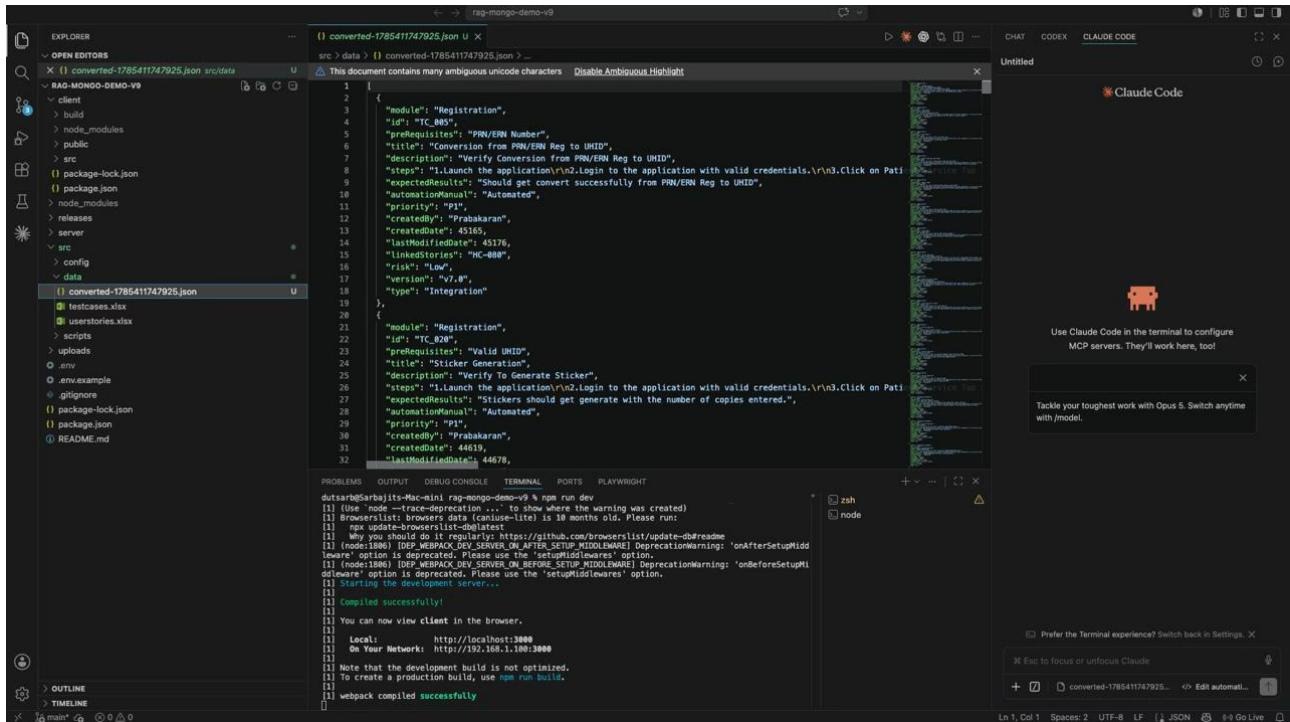
Step 1: No Existing data available under the test_cases collection



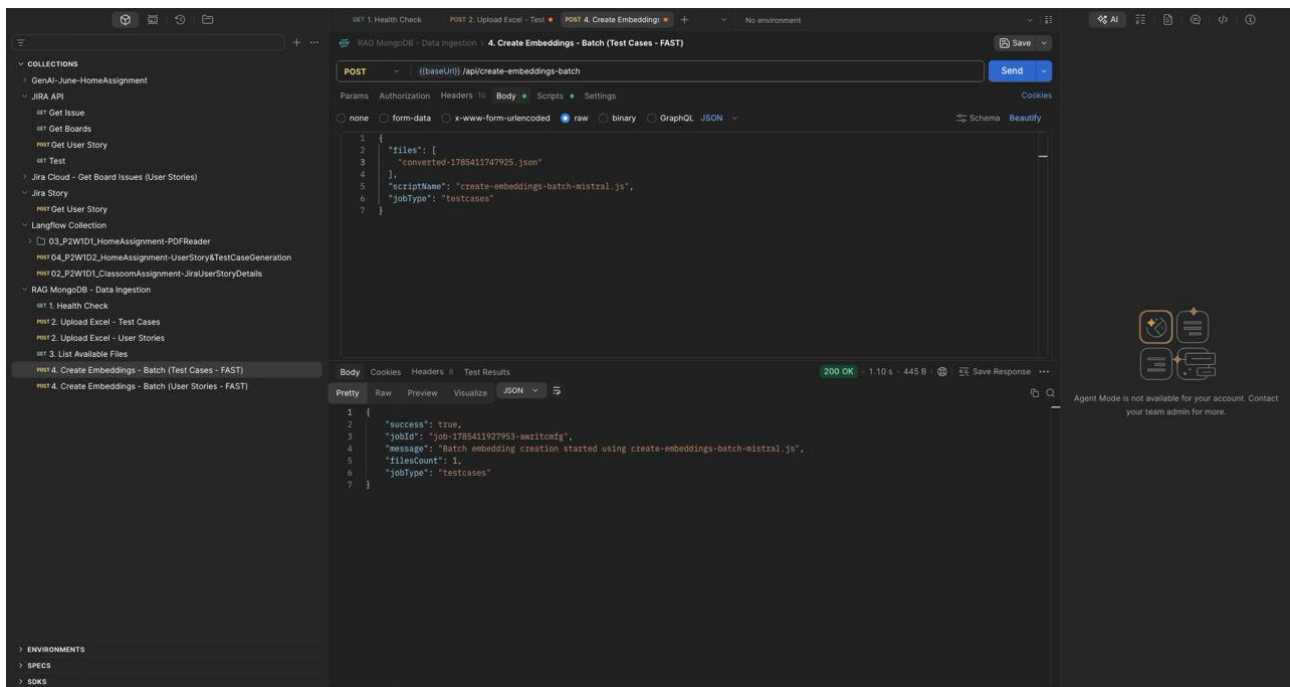
Step 2: Executed Upload Excel - Test Cases api to convert Excel test cases to JSON format



Step 3: A JSON file was created within the src->data folder within the project folder



Step 4: Executed Create Embeddings - Batch (Test Cases - FAST) api to initiate data upload to the test_cases collection



Step 5: The records are uploaded to the test_cases collection

The screenshot shows the MongoDB Atlas Data Explorer interface. On the left, the 'Data Explorer' sidebar displays the cluster 'sarbjit1210' with a tree view containing 'Gen-AI-testleaf', 'admin', 'kb_testcasegenerator', 'test_cases', 'user_stories', 'local', and 'sample_mflix'. The 'test_cases' collection is selected. The main panel shows the 'Documents' tab for 'sarbjit1210 > kb_testcasegenerator > test_cases'. It indicates 6.4K documents. A query bar contains the text 'Type a query: { field: 'value' } or [Generate query](#)'. Below the query bar are buttons for 'ADD DATA', 'BULK', and 'EXPORT CODE'. A table displays 11 records with columns: _id, module, id, prerequisites, title, and description. The bottom status bar shows 'Alert System Status: All Good' and copyright information for MongoDB, Inc. 2026.

| | _id | module | id | prerequisites | title | description |
|----|----------------------------------|----------------|----------|------------------------------|---------------------------------|-------------------|
| 1 | ObjectId('6a6b395c2035f1174...') | "Registration" | "TC_005" | "PRN/ERN Number" | "Conversion from PRN/ERN Re..." | "Verify Conve..." |
| 2 | ObjectId('6a6b395c2035f1174...') | "Registration" | "TC_020" | "Valid UHID" | "Sticker Generation" | "Verify To Ge..." |
| 3 | ObjectId('6a6b395c2035f1174...') | "Registration" | "TC_024" | "Valid UHID " | "Smart Card" | "Verify user" |
| 4 | ObjectId('6a6b395c2035f1174...') | "Registration" | "TC_027" | "Master UHID and Child UHID" | "Merge UHID" | "Verify user" |
| 5 | ObjectId('6a6b395c2035f1174...') | "Registration" | "TC_028" | "Master UHID" | "Deneger UHID" | "Verify user" |
| 6 | ObjectId('6a6b395c2035f1174...') | "Registration" | "TC_029" | "Valid UHID" | "Reissue UHID" | "Verify User" |
| 7 | ObjectId('6a6b395c2035f1174...') | "Registration" | "TC_030" | "Valid UHID" | "Reprint UHID Card" | "Verify User" |
| 8 | ObjectId('6a6b395c2035f1174...') | "Registration" | "TC_044" | "No Pre-requisite" | "Create UHID without Paymen..." | "Verify if UH..." |
| 9 | ObjectId('6a6b395c2035f1174...') | "Registration" | "TC_049" | "Valid From and to Date" | "RegStateDiscityWiseReport" | "Verify to Re..." |
| 10 | ObjectId('6a6b395c2035f1174...') | "Registration" | "TC_063" | "Valid From and to Date" | "Edit contact info" | "Verify Edit" |
| 11 | ObjectId('6a6b395c2035f1174...') | "Registration" | "TC_064" | "No Pre-requisite" | "Remarks" | "Verify Remar..." |

2. INGESTION TO *user_stories*

Step 1: No existing data available under the user_stories collection

The screenshot shows the MongoDB Atlas Data Explorer interface. On the left, the 'Data Explorer' sidebar displays the cluster 'sarbjit1210' with a tree view containing 'Gen-AI-testleaf', 'admin', 'kb_testcasegenerator', 'test_cases', 'user_stories', 'local', and 'sample_mflix'. The 'user_stories' collection is selected. The main panel shows the 'Documents' tab for 'sarbjit1210 > kb_testcasegenerator > user_stories'. It indicates 0 documents. A query bar contains the text 'Type a query: { field: 'value' } or [Generate query](#)'. Below the query bar are buttons for 'ADD DATA', 'BULK', and 'EXPORT CODE'. The main content area displays a message: 'This collection has no data. It only takes a few seconds to import data from a JSON or CSV file.' The bottom status bar shows 'Alert System Status: All Good' and copyright information for MongoDB, Inc. 2026.

Step 2: Executed Upload Excel - User Stories api, to convert Excel user stories to JSON format

The screenshot shows a REST client interface with a collection of endpoints on the left. The selected endpoint is 'POST /api/upload-excel'. The request body is a JSON object with the following parameters:

| Key | Value | Description |
|-----------|------------------|---|
| file | userstories.xlsx | Excel file containing user stories |
| dataType | userstories | Type of data: 'testcases' or 'userstories' |
| sheetName | stories | Name of the Excel sheet to convert (optional) |
| key | value | Description |

The response is a 200 OK status with a JSON body:

```
{
  "success": true,
  "message": "User stories file converted successfully",
  "outputFile": "stories-1785412329718.json",
  "output": "Converting 190 rows from sheet 'stories' into @srcs@data@src@Documents@rag-mongo-demo-v9@src@data@stories-1785412329718.json\nFiltered out: 0 empty rows\n"
```

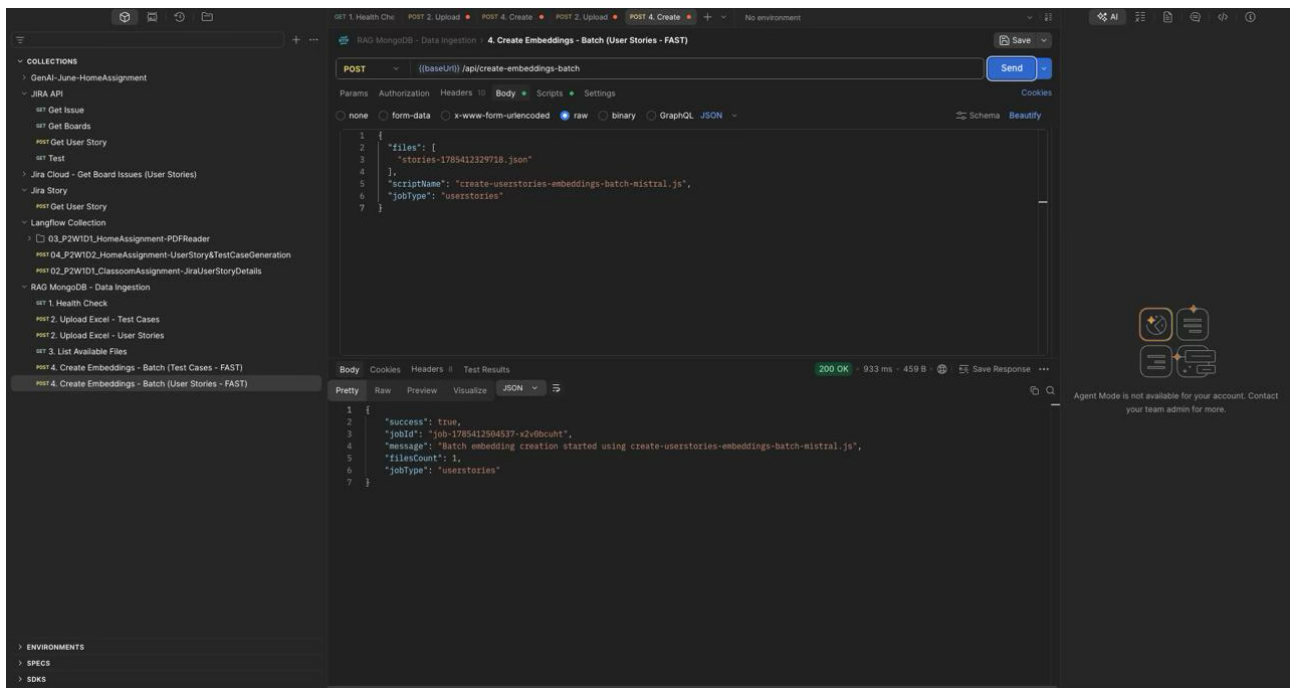
Step 3: A JSON file got created within the src->data folder within the project folder

The screenshot shows a code editor with the project structure on the left. The 'src' folder is expanded, showing the 'data' folder. The file 'stories-1785412329718.json' is selected. The contents of the file are a JSON array of user stories:

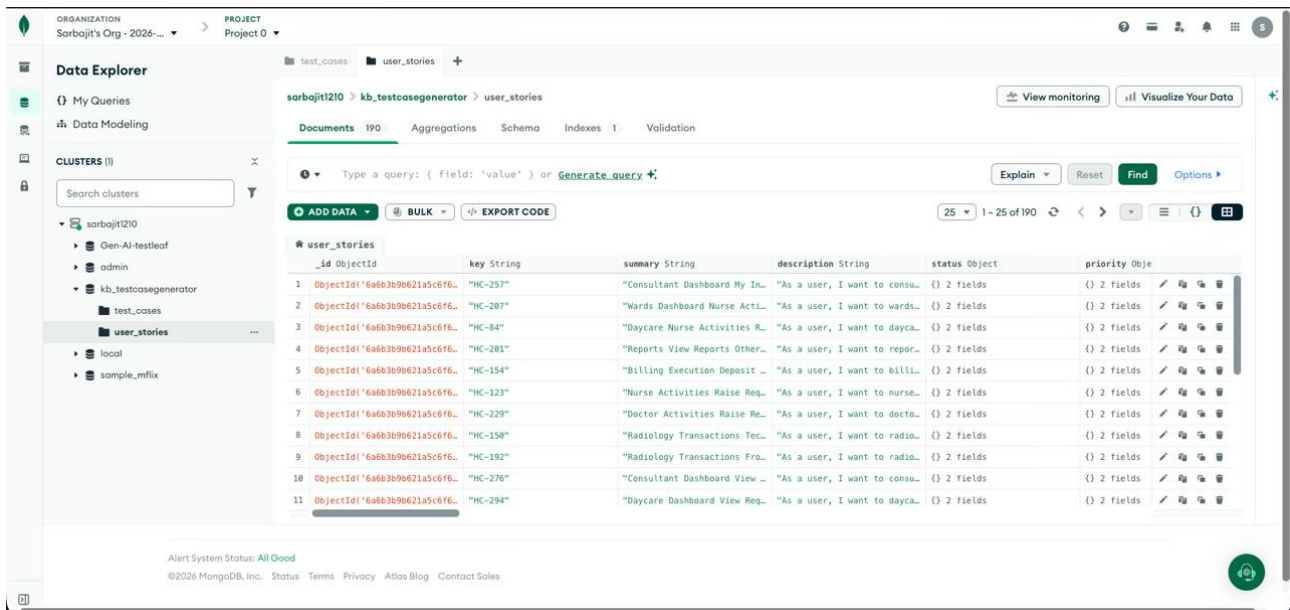
```
[
  {
    "key": "MC-257",
    "summary": "Consultant Dashboard My Inpatients",
    "description": "As a user, I want to consultant dashboard my inpatients so that the related functionality works",
    "status": {
      "name": "To Do",
      "category": "To Do"
    },
    "priority": {
      "name": "P1",
      "id": "3"
    },
    "assignee": null,
    "reporter": null,
    "created": "2022-01-06T08:00:00.000Z",
    "updated": "2026-07-30T11:52:09.828Z",
    "components": [],
    "labels": [],
    "fixVersions": [],
    "storyPoints": null,
    "project": "Health Care",
    "epic": "Emergency Management",
    "acceptanceCriteria": "Given a user with valid access, when they open the dashboard, then relevant data and",
    "businessValue": "",
    "risk": "Critical",
    "dependencies": "",
    "notes": "",
    "sprint": null,
    "team": null,
    "issueLinks": [],
    "url": "",
    "sourceType": "excel",
    "source": "MC-257"
  }
]
```

The terminal at the bottom shows the output of the batch script, indicating that the batch script completed successfully.

Step 4: Executed Create Embeddings - Batch (User Stories - FAST) api to initiate data upload to the user_stories collection



Step 5: The records are uploaded to the user_stories collection



ASSIGNMENT 3: EXPLORATORY TASK: VECTOR EMBEDDING USING COHERE AI

EMBEDDING OF SINGLE QUERY

Executed the Embed Single Text API using the embed-english-v3.0 model. This model can embed text and images with a dimension of 1024 and a max tokens (context length) of 512 tokens

The screenshot displays the Cohere Vector Embeddings API interface. The 'POST' method is selected for the endpoint `/{base_url}/v2/embed`. The request body is a JSON object:

```
{
  "model": "embed-english-v3.0",
  "texts": [
    "The quick brown fox jumps over the lazy dog"
  ],
  "input_type": "search_document",
  "embedding_types": ["float"]
}
```

The response body shows a large array of embedding values, indicating a successful request. The status is 200 OK.

EMBEDDING OF MULTIPLE QUERIES

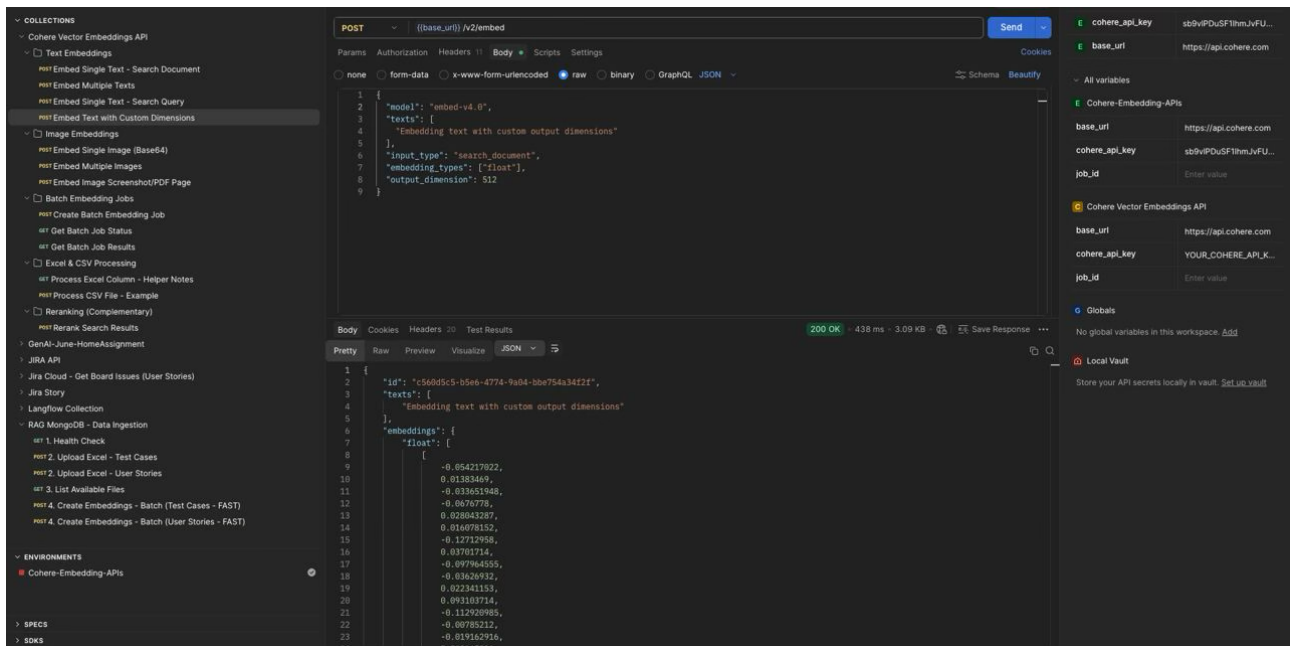
Executed the Embed Multiple Texts API to vector-embed multiple lines of text. The model used for this API call is embed-english-v3.0

The screenshot displays the Cohere Vector Embeddings API interface. The 'POST' method is selected for the endpoint `/{base_url}/v2/embed`. The request body is a JSON object:

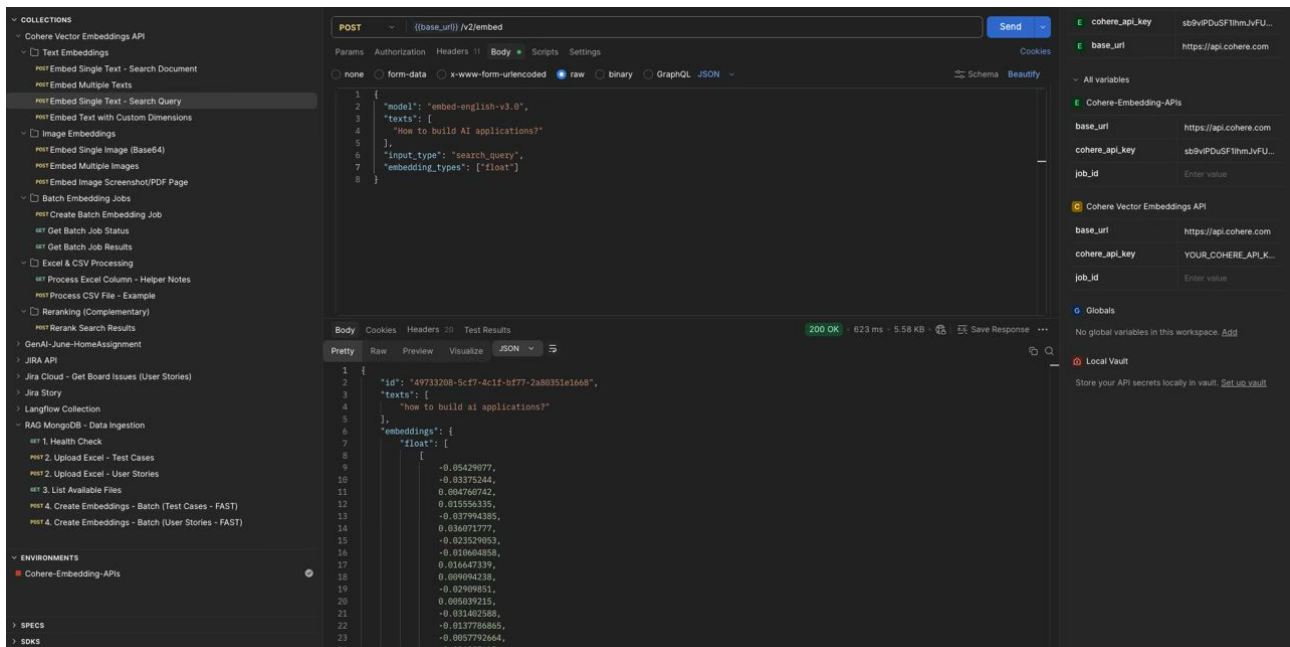
```
{
  "model": "embed-english-v3.0",
  "texts": [
    "Machine learning is a subset of artificial intelligence",
    "Natural language processing helps computers understand human language",
    "Vector embeddings represent text in high-dimensional space"
  ],
  "input_type": "search_document",
  "embedding_types": ["float"]
}
```

The response body shows a large array of embedding values, indicating a successful request. The status is 200 OK.

Embed a single line of text using search_query as the input_type field. The model used is embed-english-v3.0. This type of API call is done while embedding a search query made against an existing vector DB to perform a search



Embedding text with custom output dimension using “output_dimension” parameter. This parameter is applicable only for v4.0 models



COLLECTIONS

- Cohere Vector Embeddings API
- Text Embeddings
 - Embed Single Text - Search Document
 - Embed Multiple Texts
 - Embed Single Text - Search Query
 - Embed Text with Custom Dimensions
- Image Embeddings
 - Embed Single Image (Base64)
 - Embed Multiple Images
 - Embed Image Screenshot/PDF Page
- Batch Embedding Jobs
 - Create Batch Embedding Job
 - Get Batch Job Status
 - Get Batch Job Results
- Excel & CSV Processing
 - Process Excel Column - Helper Notes
 - Process CSV File - Example
- Reranking (Complementary)
 - Rerank Search Results

POST

[[base_url]]/v2/embed

Send

Params

Authorization

Headers

Body

Scripts

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL JSON

Schema

Beauty

```

1 {
2   "model": "embed-v4.0",
3   "images": [
4     "data:image/png;base64,iVBORw0KG0AAAAASU1EIjAa1AAAEAAACAAKX7B0AAAAP19WEUjwTKi//5/57s:
5     P18E8A5F9m59m30wC629zydydgwF7Nuc448c4uG00u1a1qTACyU0V8AL1YAC4+qE9NA81u01EqVR42y0cSLcNpFyS+
6     21TU0onh21986uE0U07
7     t5YH84JREEREG01W8q5HER6F1G0W1111gg6Z2j3kHRD1kyL5WpJk1V4p4w7S4wF6D01uZCjwP0W86G83mC4wG01S3G0RVD0eaZCwR7J54
8     7656VqN6WV4Hd4+U4v75tKtC00a2004G05P747HqVIA
9     10J02pDg4wF7P5S019W2B4d44N2wF75C04v0tXm1t11gW6W4wP0P9V7Y41Aq211v7T0F1u11F7H465323aW5P5L2yP0C21225x
10    M4y5f63Q2ClYx8u6W4W7Y0sd18ku0Dm0t1NVEV4wJ384vYwN4wHeddy2E2W24k017H411b4327MM4B0Vg1S106C70G5S9F+570282w0a01kg
11    +cY7V4u15T7F7Y27PRL7yqncp6eL1JhM1JnF24h1MOYc124CyF0eLdGP3W62GFMstQ47CWh07q0wW7TUJk0Z7Vg8u5pH5B0k4Gq4x445S2131/
12    xLDj0XtUPID1M4WV51S4S1Jy5YF7yhoF78SL6NDRH24W9K0W
13    +D0136011Y2h76e0tF1400142W60009V1Lx+4W00W27316
14    1b04k7N0C21u18w4wF722920000W12E1u04h374
15    +22Q7qW8W030T2gzzecht40P2Kw6L6Gfn1jP07B7M73170P73Gy57G242j05uH9Z6ELKX7C2N2G0uW01cm6G0znd5w/
16    X1xR612aw0G5K91anz736G8N8u42D7HfK0pC04063026w0Q0uXZNDU0H33wC7K03jgacyK7YnpD0H9VH5V2G7962H0A2131EWP7V197357G3L
17    45148u51576h0V7D0wFg51A7J2+Y00477P25CV45Z2F10GK4V
18    +W0Q1H4h1u7p1u1a1k4Kup0n0C20W16P4P04D0u7JW+070p0C4c4643C6K4K4HJY3/0
19    H0V9W47G2P7Ythm5v4wA4A047b9P0P7150u0271x1312w0H0K4w+M070m0u01P41R0V4pV7S04h1w4X7n51EP414w707W0V0u099W47w41act3h1/
20    H0V9W47G2P7Ythm5v4wA4A047b9P0P7150u0271x1312w0H0K4w+M070m0u01P41R0V4pV7S04h1w4X7n51EP414w707W0V0u099W47w41act3h1/

```

200 OK

387 ms

6.25 KB

Save Response

Body

Cookies

Headers

Test Results

2/2

Pretty

Raw

Preview

Visualize

JSON

```

1 {
2   "id": "48389f99-362d-4854-b70d-48d3bc4153c",
3   "images": [
4     {
5       "width": 588,
6       "height": 896,
7       "format": "image/png",
8       "bit_depth": 4
9     }
10  ],
11  "embeddings": [
12    "embedding": [
13      [
14        -0.922386411,
15        0.83004803,
16        -0.94399243,
17        -0.009788385,
18        0.609712585,
19        -0.06402381,
20        -0.009648748,
21        -0.01739004,
22        0.004628616,
23        0.029744549,

```

Variables in request

- cohere_api_key: sb9vP0u5F1hmJvFU...
- base_url: https://api.cohere.com

All variables

- Cohere-Embedding-APIs
- base_url: https://api.cohere.com
- cohere_api_key: sb9vP0u5F1hmJvFU...
- job_id: Enter value

- Cohere Vector Embeddings API
- YOUR_COHERE_API_K...: https://api.cohere.com

job_id: Enter value

Globals

No global variables in this workspace. Add

Local Vault

Store your API secrets locally in vault. [Set up vault](#)

COLLECTIONS

Cohere Vector Embeddings API

Text Embeddings

Embed Single Text - Search Document

Embed Multiple Texts

Embed Single Text - Search Query

Embed Text with Custom Dimensions

Image Embeddings

Embed Image (Base64)

Embed Multiple Images

Embed Image Screenshot/PDF Page

Batch Embedding Jobs

Create Batch Embedding Job

Get Batch Job Status

Get Batch Job Results

Excel & CSV Processing

Process Excel Column - Helper Notes

Process CSV File - Examples

Ranking (Complementary)

ReRank Search Results

https://api.cohere.com/v2/embed

GenAI - User Onboard Assignment

JIRA API

Jira Cloud - Get Board Issues (User Stories)

Jira Story

Langflow Collection

RAG MongoDB - Data Ingestion

1. Health Check

2. Upload Excel - Text Cases

3. List Available Files

4. Create Embeddings - Batch (Test Cases - FAST)

4.1. Create Embeddings - Batch (User Stories - FAST)

ENVIRONMENTS

Cohere-Embedding-APIs

SPECS

DOCS

POST

[(base_url) /v2/embed]

Send

Params

form-data

x-www-form-urlencoded

raw

binary

GraphQL JSON

Body

Scripts

Settings

cookies

Schema

Beautify

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

341

342

343

344

345

346

347

348

349

350

351

352

353

354

355

356

357

358

359

360

361

362

363

364

365

366

367

368

369

370

371

372

373

374

375

376

377

378

379

380

381

382

383

384

385

386

387

388

389

390

391

392

393

394

395

396

397

398

399